SQL: Joins and Its Types

Utkarsh Sahu

Real-world data is rarely stored in one giant table. Instead, to avoid repetition and keep things organized, it's split into multiple, related tables. For example, you might have one table for `Customers` and another for their `Orders`.

SQL Joins are the powerful mechanism that lets you temporarily connect these separate tables based on a related column. They are the "glue" that brings your relational database to life.

- You can't find out which customer placed a specific order without connecting the `Customers` and `Orders` tables. Joins allow you to ask meaningful questions by combining information from two or more tables. For example, "Show me the names of all customers who ordered a specific product."
- **The "Relational" in RDBMS**: Joins are the very heart of a **R**elational **D**atabase **M**anagement **S**ystem. Without them, you'd just have a collection of isolated lists. Different types of joins (`INNER`, `LEFT`, `RIGHT`) give you precise control over *how* you combine this information.

1. Natural Join

A `NATURAL JOIN` is the simplest looking join. It automatically finds all columns that have the **exact same name** in both tables and matches rows where the values in those columns are equal. It then presents only one copy of these shared columns in the final result.

Let's imagine two tables: `Customers` and `Orders`. Both have a column named `customer_id`.

Example: List the names of customers along with the dates of the orders they placed.

```
-- Using an older, implicit join syntax (less readable)
SELECT name, order_date
FROM Customers, Orders
WHERE Customers.customer_id = Orders.customer_id;

-- Same query using the modern "NATURAL JOIN" construct
SELECT name, order_date
FROM Customers
NATURAL JOIN Orders;
```

The Danger of Natural Joins

What if both the `Customers` table and the `Orders` table also have a column named `last_updated`? A `NATURAL JOIN` will try to match rows based on **both** `customer_id` AND `last_updated`.

- `... WHERE Customers.customer_id = Orders.customer_id AND Customers.last_updated = Orders.last_updated;`

This is almost never what you want and will incorrectly filter out valid results. Because of this unpredictability, most developers avoid `NATURAL JOIN` in professional code.

2. Inner Join

An `INNER JOIN` is the most common and explicit type of join. It does the same thing as a natural join—finds matching rows—but it forces you to state the exact matching condition using the **`ON`** clause. This makes your query clear, predictable, and safe from accidental matches.

Example: Let's correctly get a list of customer names and their order dates.

```
SELECT
  Customers.name,
  Orders.order_date
FROM
 Customers INNER JOIN Orders
 ON Customers.customer_id = Orders.customer_id;
```

This query is crystal clear: it will **only** join based on `customer_id`, regardless of any other columns with the same name. It finds the intersection of the two tables based on the rule you provide. If a customer has no orders, they won't appear in the result.

3. Outer Join

What if you want to see all customers, **including those who haven't placed an order yet**? An `INNER JOIN` would exclude them. This is where `OUTER JOIN` comes in. It prevents the loss of information by keeping non-matching rows from one or both tables and using `NULL` as a placeholder for the missing data.

3.1. Left (Outer) Join

This join **returns all rows from the left table** (`Customers`), and the matched rows from the right table (`Orders`). If a customer has no matching order, their order information will be shown as `NULL`.

Example: List all customers and any orders they may have.

```
SELECT
  Customers.name,
  Orders.order_date
FROM Customers
LEFT OUTER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

The result will include customers with `NULL` for `order_date`, showing they've never ordered.

3.2. Right (Outer) Join

This join **returns all rows from the right table** and any matched rows from the left. It's less common, but useful in some scenarios.

Example: List all orders and the customer who placed them. (This would also show any "orphan" orders that might exist in the database without a valid customer attached).

```
SELECT
  Customers.name,
  Orders.order_date
FROM Customers
```

```
RIGHT OUTER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

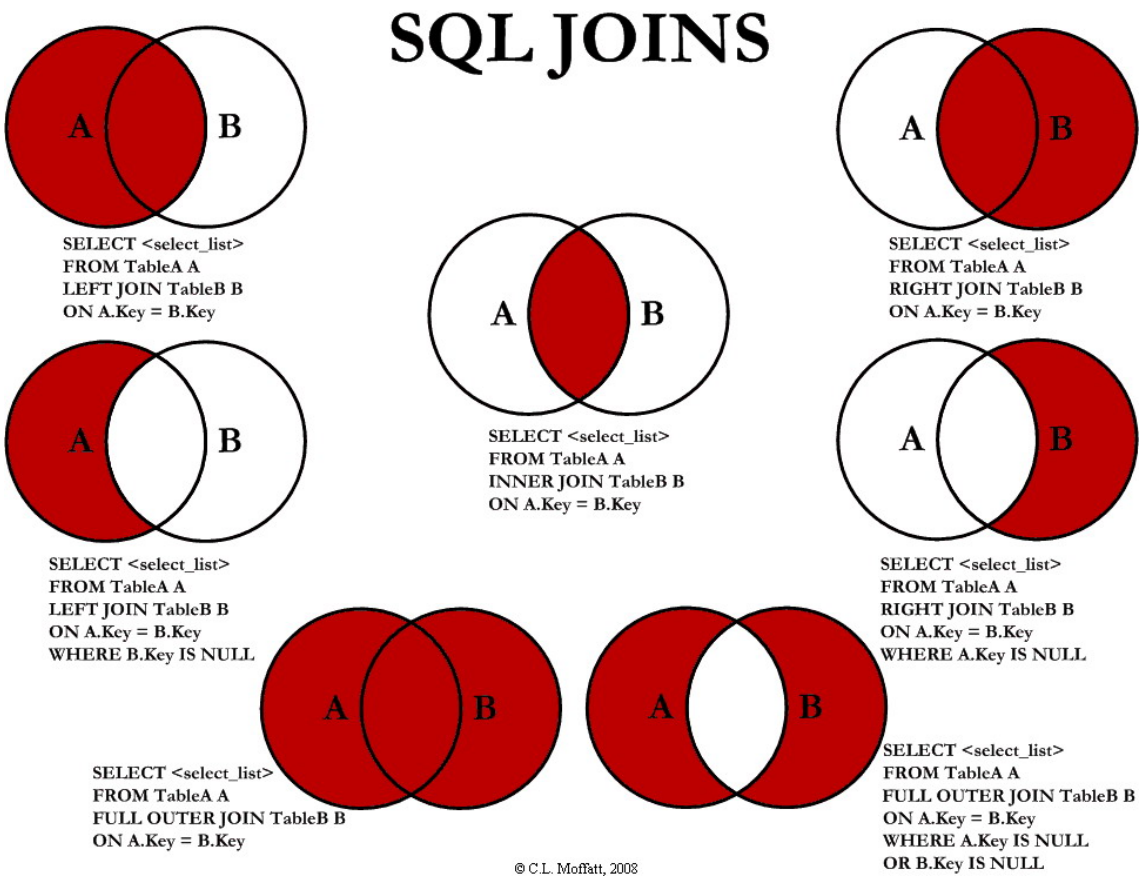
3.3. Full Outer Join

This returns **all rows from both tables**. It places `NULL` on the left side when there is no match in the right table, and `NULL` on the right side when there is no match in the left table. It gives you the complete picture of both tables combined.

Example: List all customers and all orders, showing all connections between them. This comprehensive list will include:

- Customers who have never placed an order.
- "Orphan" orders in the system that are not linked to a valid customer.

```
SELECT
  Customers.name,
  Orders.order_date
FROM Customers
FULL OUTER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```



. Types of SQL JOINS

4. Can we do a self Join?

Yes we can join a table on itself, to see some interesting results. For example, Imagine you have a single table called `Employees`. This table lists all employees and, for each one, it also stores the ID of their direct manager. The key is that the manager is also an employee in the very same table.

The Challenge: How do you create a report that lists each employee's name next to their manager's name? The `manager_id` column only gives us a number, not the actual name.

Here's what the `Employees` table might look like:

employee_id	name	manager_id
101	Sarah	103
102	David	103
103	Julia	104
104	Robert	NULL
105	Peter	104

The Solution: The Self-Join

To solve this, you join the `Employees` table to **itself**. You pretend you have two copies of the table and give them different names (aliases) to tell them apart.

- One copy will represent the **Employee** (we'll call it `e`).
- The other copy will represent the **Manager** (we'll call it `m`).

```
SELECT
    e.name AS employee_name,  -- The employee's name from the 'e' copy
    m.name AS manager_name   -- The manager's name from the 'm' copy
FROM
    Employees e  -- First copy of the table, aliased as 'e' for Employee
INNER JOIN
    Employees m  -- Second copy of the table, aliased as 'm' for Manager
ON
    e.manager_id = m.employee_id; -- The crucial link!
```

The query would produce the following clear and readable report:

employee_name	manager_name
Sarah	Julia
David	Julia
Julia	Robert
Peter	Robert